

SIEM LAB BUILD REPORT

Build By:	Ashogbon Ayomikun Abdulazeez
Date:	30 th Of July 2026

Table of Contents

- Executive Summary
- Scope & Objective
- Methodology
 - Lab Architecture
 - SIEM Platform Selection
 - Architecture Design
 - Lab Segmentation
 - Network Layout
 - Wazuh Installation and Initial Configuration
 - Initial Deployment Approach
 - Docker Environment Setup
 - Security Configuration
 - Resource Optimization
 - Service Deployment and Verification
 - Dashboard Access
 - Ubuntu Endpoint Integration
 - Agent Deployment
 - Agent Installation
 - Agent Verification
 - Log Validation
 - Suricata Integration
 - Suricata Installation
 - Configuration
 - Integration with Wazuh
 - Validation
 - VirusTotal Integration
 - VirusTotal Configuration
 - Custom Detection Rules
 - Validation
 - Active Response
 - Automated Threat Response
 - Validation
 - Windows Endpoint Integration
 - Wazuh Agent Deployment
 - Agent Installation and Verification
 - Sysmon Installation
 - Sysmon Configuration
 - Log Validation
 - pfSense Firewall Integration
 - pfSense Configuration
 - Wazuh Configuration
 - Log Parsing
 - Validation

- Dashboard Development
 - Authentication Activity Dashboard
 - Endpoint Process Activity Dashboard
 - Network Traffic Overview Dashboard
 - Custom Detection Rules
 - Brute-Force Login Detection Rule
 - Suspicious Process Execution Rule
- Findings
 - Authentication Activity Dashboard
 - Endpoint Process Activity Dashboard
 - Network Traffic Overview Dashboard
 - Custom Correlation Rules
 - Brute-Force Login Detection Rule
 - Suspicious Process Execution Rule
- Gaps and Challenges
- Limitations
- Recommendations and Next Steps
- Appendix
 - A. Custom Wazuh Correlation Rules (`local_rules.xml`)
 - B. Custom pfSense Decoder (`local_decoder.xml`)
 - C. pfSense Firewall Detection Rule
 - D. Windows Agent Configuration
 - E. Sysmon Installation
 - F. pfSense Remote Syslog Configuration
 - G. Wazuh Docker Remote Syslog Configuration

Executive Summary

I built a home SOC (Security Operations Center) lab to practice detecting and responding to real cyber attacks, the same core work a Tier 1 SOC analyst does on the job. At the center is Wazuh, which provides a single dashboard to monitor everything happening across the lab in real time. I integrated Suricata to monitor network traffic for suspicious activity and connected VirusTotal so suspicious files and IPs are automatically checked against a global threat database instead of relying on guesswork. When VirusTotal confirms a file is malicious, an automated response script immediately deletes it without waiting for manual intervention. I also created custom detection rules to identify brute-force login attempts and programs executing from suspicious locations, ensuring these threats are automatically flagged and handled instead of getting buried in raw logs. This project demonstrates hands-on SOC skills in building detection systems, automating threat response, and troubleshooting detection rules until they work as intended.

Scope & Objective

The goal of this project was to build a functional SOC home lab from scratch that could detect real attack behavior, not just collect logs. Rather than relying solely on Wazuh's default rules, the objective was to design custom detection logic for brute-force login correlation and suspicious process execution, then pair it with automated response so confirmed threats are acted on immediately instead of sitting in a queue for manual review. The scope covered the entire detection and response pipeline, including log collection (Sysmon, pfSense syslog, and Suricata), centralized analysis and alerting (Wazuh), threat intelligence enrichment (VirusTotal), and active response (automated file removal), essentially replicating, on a small scale, how a real SOC's detection and response workflow operates.

Methodology

Lab Architecture

SIEM Platform Selection

The first step was selecting the SIEM platform for the SOC lab. Wazuh was chosen because it provides the Manager, Indexer, Dashboard, and endpoint agents in a single integrated solution.

Architecture Design

The lab architecture was created using draw.io. Additional shape libraries were enabled to access the required networking and computer icons. The architecture was then built using three computer icons, one server icon, one firewall icon, and one cloud icon.

Lab Segmentation

Two sections were created to organize the environment:

- **SOC Environment**
- **Endpoint Environment**

The Wazuh server (Kali Linux) was placed in the **SOC Environment**, while the Windows virtual machine and Ubuntu virtual machine were placed in the **Endpoint Environment**. The firewall was positioned between both sections to represent its role in controlling communication between the environments. A cloud icon was added above the firewall to represent the WAN connection.

Network Layout

A bidirectional connection was drawn between the WAN and the firewall to illustrate network communication. The remaining components were then connected according to the planned architecture, and each device was labeled with its hostname, IP address, and role within the SOC lab.

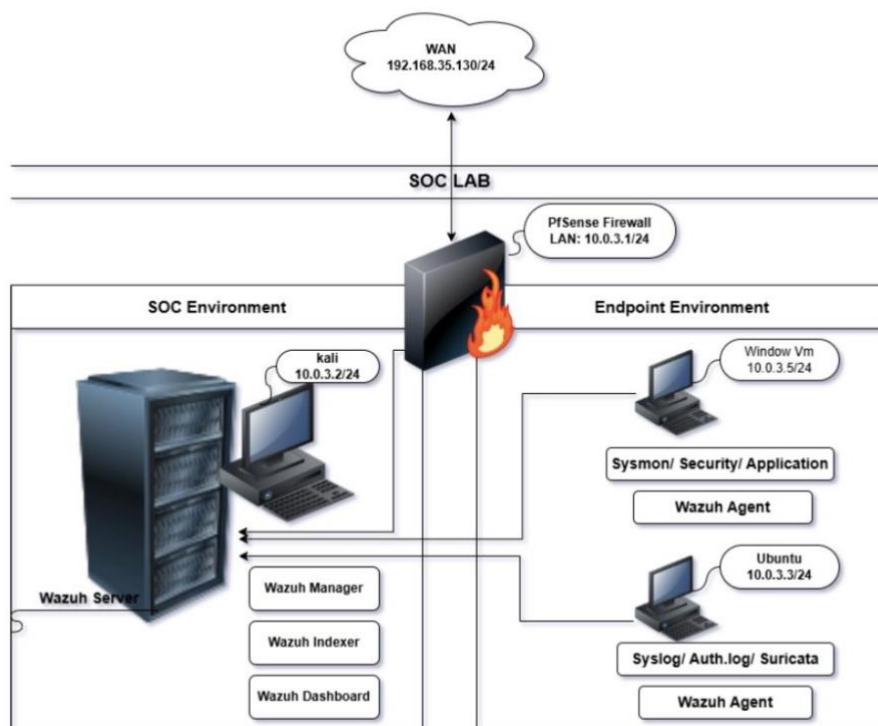


Figure 1: SOC Home Lab Architecture created using draw.io.

Wazuh Installation and Initial Configuration

Initial Deployment Approach

The initial approach was to install Wazuh using the standard installation method. However, repeated installation failures prevented the deployment from completing successfully. After researching alternative deployment methods, the official Wazuh Docker deployment was selected and used to successfully install the platform.

Docker Environment Setup

Docker and Docker Compose were installed to provide the container environment required to run Wazuh. The official Wazuh Docker repository was then downloaded, providing the configuration files needed to deploy the Wazuh Manager, Wazuh Indexer, and Wazuh Dashboard as containers.

Security Configuration

Before deploying the containers, self-signed SSL/TLS certificates were generated to secure communication between the Wazuh components. In addition, the Linux memory mapping limit was increased to meet Wazuh's recommended system requirements and ensure stable operation.

Resource Optimization

Because the Kali Linux virtual machine was allocated **4 GB of RAM**, memory limits were configured for each Docker container to prevent excessive resource consumption. The Wazuh Manager and Dashboard were each limited to **512 MB**, while the Wazuh Indexer was allocated **1 GB**. This ensured that all Wazuh services operated reliably within the available system resources.

Service Deployment and Verification

After the resource limits were configured, the Wazuh containers were started. The deployment was verified by confirming that the Wazuh Manager, Wazuh Indexer, and Wazuh Dashboard were all running successfully.

Dashboard Access

Finally, the Wazuh Dashboard was accessed through a web browser using the server's local host address or IP address. The default administrator credentials provided by the Docker deployment were used to log in, confirming that the SIEM platform had been successfully deployed and was ready for endpoint onboarding.

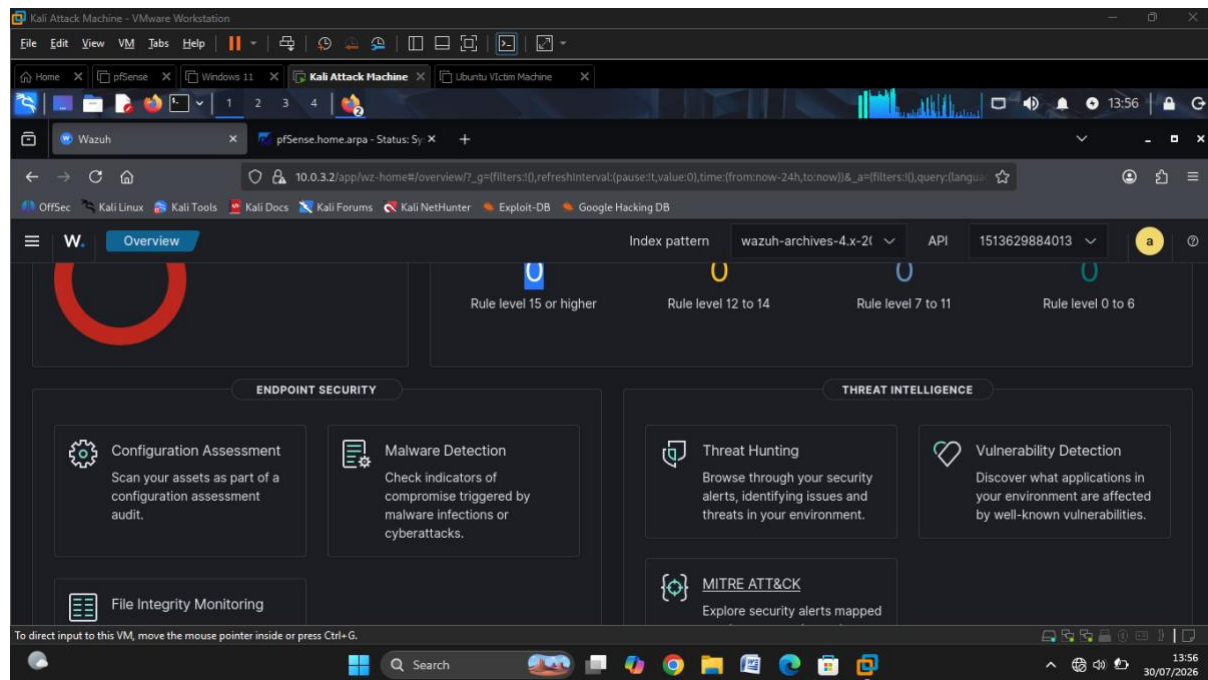


Figure 2: *Successful Wazuh Dashboard Deployment.*

Ubuntu Endpoint Integration

Agent Deployment

The Ubuntu virtual machine was selected as the first monitored endpoint. From the Wazuh Dashboard, the **Deploy New Agent** option was used to generate the Linux agent installation package. During the deployment process, the Wazuh server's IP address (**10.0.3.1**) was specified, and the agent was assigned the name **Hayzed**.

Agent Installation

The generated installation package was executed on the Ubuntu virtual machine. Once the installation was complete, the Wazuh agent service was started and enabled to ensure it automatically started whenever the system was powered on.

Agent Verification

After the agent was started, the Wazuh Dashboard was checked to confirm that the Ubuntu endpoint had successfully registered with the Wazuh Manager. The agent appeared with an **Active** status, confirming that communication between the endpoint and the SIEM server had been established successfully.

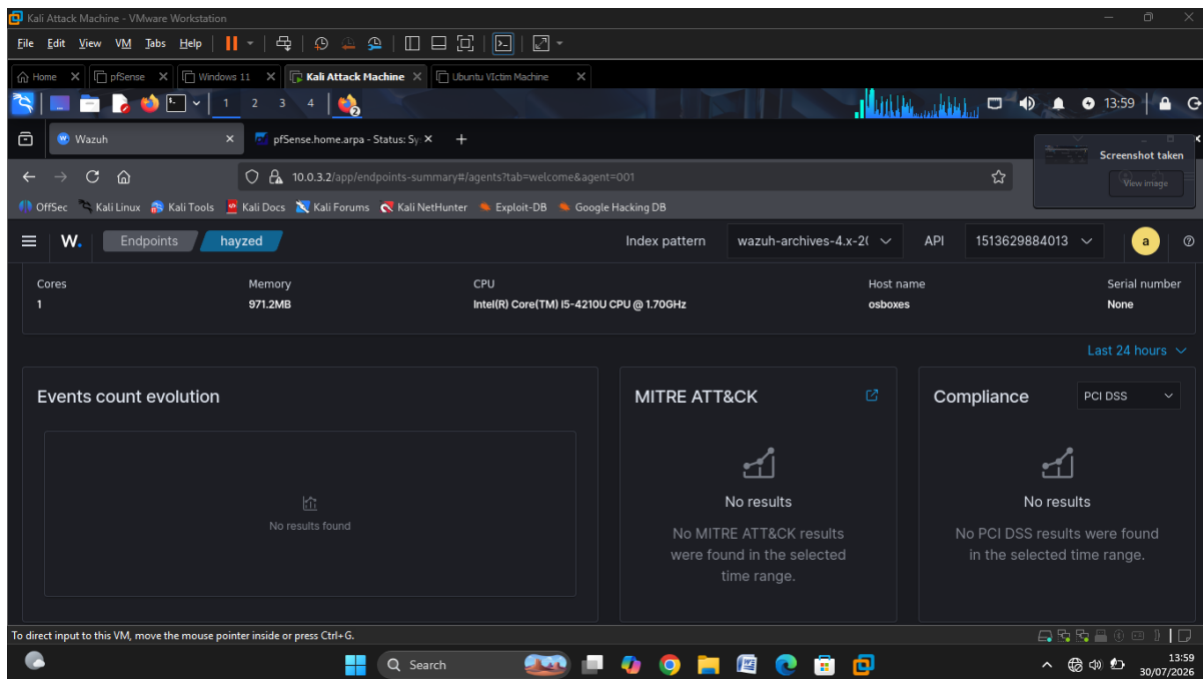


Figure 3: *Ubuntu endpoint successfully registered as an active Wazuh agent.*

Log Validation

To verify that the endpoint was forwarding logs correctly, multiple failed and successful ssh login attempts were intentionally generated on the Ubuntu virtual machine. Wazuh successfully detected the authentication and generated corresponding alerts in the dashboard, confirming that log collection and alert generation were functioning as expected.

Timestamp	Agent	Event Description	Alert ID
Jul 23, 2026 @ 00:53:43.921	hayzed	PAM: Login session opened.	5501
Jul 23, 2026 @ 00:53:43.920	hayzed	Successful sudo to ROOT executed.	5402
Jul 23, 2026 @ 00:53:43.106	hayzed	First time user executed sudo.	5403
Jul 23, 2026 @ 00:53:43.101	hayzed	PAM: Login session opened.	5501
Jul 23, 2026 @ 00:53:43.100	hayzed	PAM: Login session opened.	5501
Jul 23, 2026 @ 00:53:30.572	hayzed	PAM: User login failed.	5503
Jul 23, 2026 @ 00:53:29.751	hayzed	PAM: User login failed.	5503
Jul 23, 2026 @ 00:53:21.689	hayzed	PAM: Login session opened.	5501
Jul 23, 2026 @ 00:53:21.689	hayzed	sshd: authentication success.	5715
Jul 23, 2026 @ 00:53:21.689	hayzed	sshd: authentication failed.	5760
Jul 23, 2026 @ 00:53:21.689	hayzed	sshd: authentication success.	5715
Jul 23, 2026 @ 00:53:21.689	hayzed	PAM: Login session opened.	5501

Figure 4: *Authentication failure alerts generated from the Ubuntu endpoint.*

Suricata Integration

Suricata Installation

Suricata was installed on the Ubuntu endpoint to provide network intrusion detection capabilities. The latest stable release and its accompanying detection rules were installed to enable the identification of suspicious network activity.

Configuration

After installation, Suricata was configured to monitor the Ubuntu network interface and inspect network traffic. The default configuration was updated to define the monitored network, specify the rule location, and ensure that the correct network interface was used for packet capture. During configuration, compatibility issues related to certain application-layer protocols were resolved by enabling the required protocol detection settings. The configuration was then validated to ensure that Suricata could load successfully without errors.

Integration with Wazuh

To centralize network security monitoring, the Wazuh agent was configured to collect Suricata's JSON event logs and forward them to the Wazuh Manager. This enabled Suricata alerts to be displayed alongside other endpoint events within the Wazuh Dashboard.

Validation

To verify the integration, network traffic was intentionally generated from another system within the lab. Suricata successfully detected the activity and forwarded the corresponding alerts to Wazuh, where they were visible through the **Threat Hunting** dashboard.

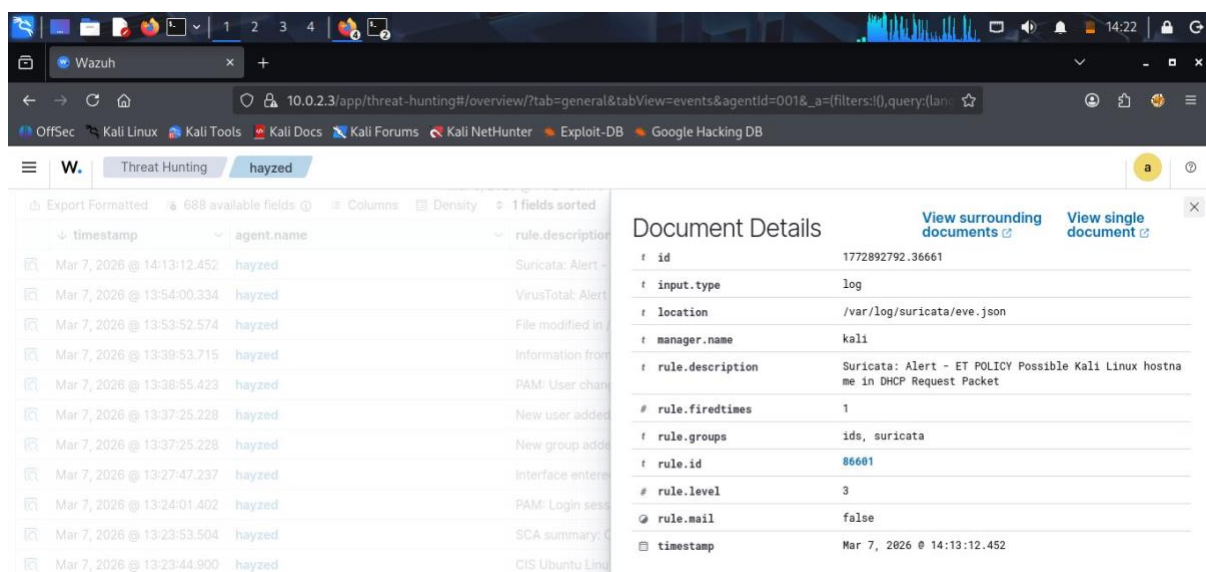


Figure 5: Suricata alerts displayed in the Wazuh Dashboard.

VirusTotal Integration

VirusTotal Configuration

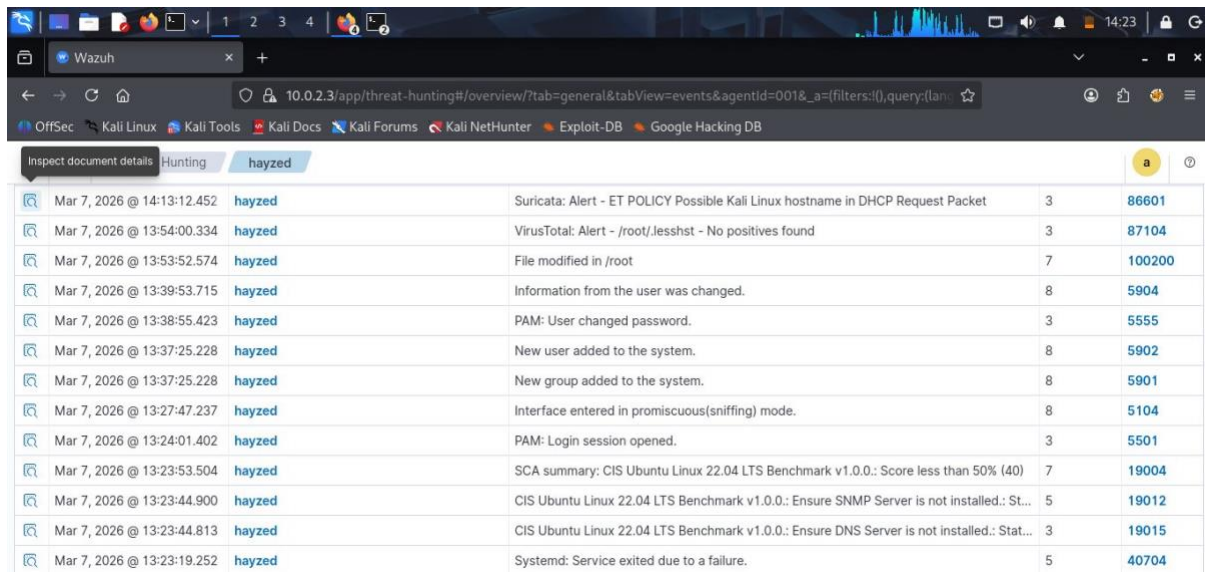
VirusTotal was integrated with Wazuh to enrich security alerts with external threat intelligence. An API key obtained from a VirusTotal account was configured within Wazuh, allowing the platform to automatically query VirusTotal whenever selected security events occurred.

Custom Detection Rules

Custom file monitoring rules were created to detect files that were added to or modified within high-risk directories, including user home directories and temporary folders. When these events occurred, Wazuh automatically submitted the file information to VirusTotal for reputation analysis.

Validation

The integration was verified by generating file creation and file modification events within the monitored directories. Wazuh successfully queried VirusTotal and displayed the reputation results within the corresponding security alerts, providing additional context for incident analysis.



Timestamp	Alert Description	Score	VirusTotal ID
Mar 7, 2026 @ 14:13:12.452	Suricata: Alert - ET POLICY Possible Kali Linux hostname in DHCP Request Packet	3	86601
Mar 7, 2026 @ 13:54:00.334	VirusTotal: Alert - /root/.lessht - No positives found	3	87104
Mar 7, 2026 @ 13:53:52.574	File modified in /root	7	100200
Mar 7, 2026 @ 13:39:53.715	Information from the user was changed.	8	5904
Mar 7, 2026 @ 13:38:55.423	PAM: User changed password.	3	5555
Mar 7, 2026 @ 13:37:25.228	New user added to the system.	8	5902
Mar 7, 2026 @ 13:37:25.228	New group added to the system.	8	5901
Mar 7, 2026 @ 13:27:47.237	Interface entered in promiscuous(sniffing) mode.	8	5104
Mar 7, 2026 @ 13:24:01.402	PAM: Login session opened.	3	5501
Mar 7, 2026 @ 13:23:53.504	SCA summary: CIS Ubuntu Linux 22.04 LTS Benchmark v1.0.0.: Score less than 50% (40)	7	19004
Mar 7, 2026 @ 13:23:44.900	CIS Ubuntu Linux 22.04 LTS Benchmark v1.0.0.: Ensure SNMP Server is not installed.: St...	5	19012
Mar 7, 2026 @ 13:23:44.813	CIS Ubuntu Linux 22.04 LTS Benchmark v1.0.0.: Ensure DNS Server is not installed.: Stat...	3	19015
Mar 7, 2026 @ 13:23:19.252	Systemd: Service exited due to a failure.	5	40704

Figure 6: *VirusTotal threat intelligence integrated with Wazuh alerts*

Active Response

Automated Threat Response

To reduce the need for manual intervention, Wazuh Active Response was configured to automatically remove files that VirusTotal identified as malicious. A custom response script was created and linked to the VirusTotal detection rule, enabling the system to respond immediately whenever a malicious file was confirmed.

Validation

The automated response was tested using a file that triggered the VirusTotal detection rule. Once the file was identified as malicious, the active response mechanism executed automatically and removed the file from the endpoint. The action was recorded in the Wazuh logs, confirming that the response had completed successfully.

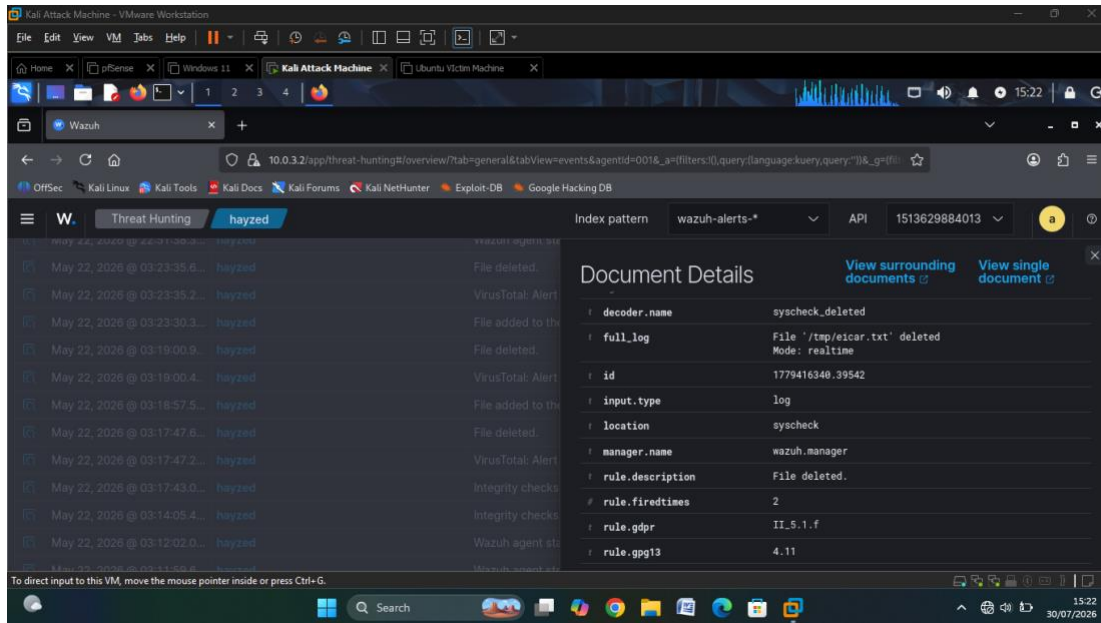


Figure 7: Automatic file removal triggered by Wazuh Active Response.

Windows Endpoint Integration

Wazuh Agent Deployment

The Windows virtual machine was onboarded as the second monitored endpoint. Using the **Deploy New Agent** option in the Wazuh Dashboard, the Windows agent installation package was generated. During the deployment process, the Wazuh server's IP address (**10.0.3.2**) was specified, and the endpoint was assigned the name **Hayzed-Windows-VM**.

Agent Installation and Verification

The generated installation package was executed on the Windows virtual machine with administrative privileges. Once the installation was complete, the Wazuh agent service was started. The Wazuh Dashboard was then checked to verify that the Windows endpoint had successfully connected to the Wazuh Manager. The agent appeared as **Active**, confirming successful communication between the endpoint and the SIEM server.

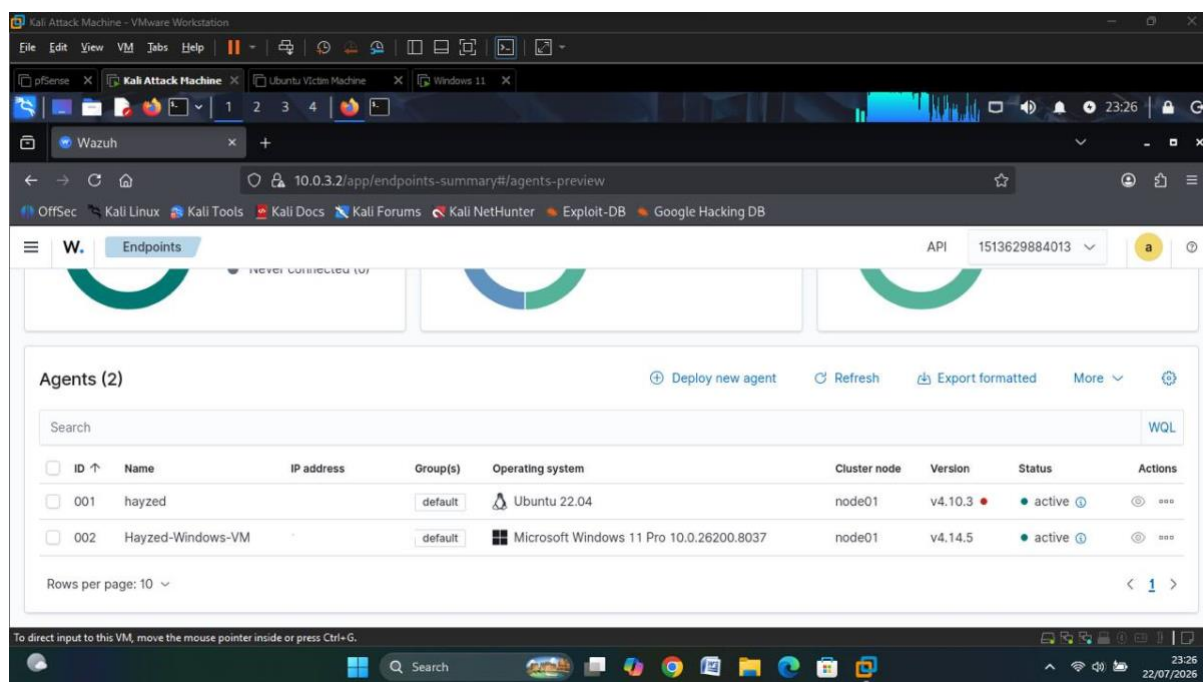


Figure 8: Windows endpoint successfully registered as an active Wazuh agent.

Sysmon Installation

To enhance Windows event logging, **Sysmon (System Monitor)** was installed on the Windows virtual machine. After installation, the Sysmon service was verified to ensure it was running correctly.

Sysmon Configuration

A community-maintained Sysmon configuration based on the **SwiftOnSecurity** baseline was applied to improve visibility into system activity, including process creation, network connections, and other security-relevant events. The Wazuh agent configuration was then updated to collect Sysmon event logs from the Windows Event Log and forward them to the Wazuh Manager.

Log Validation

To verify that the integration was functioning correctly, normal user activity was performed on the Windows virtual machine. Wazuh successfully collected the Windows and Sysmon events, including process creation events, confirming that the Windows endpoint was forwarding logs correctly.

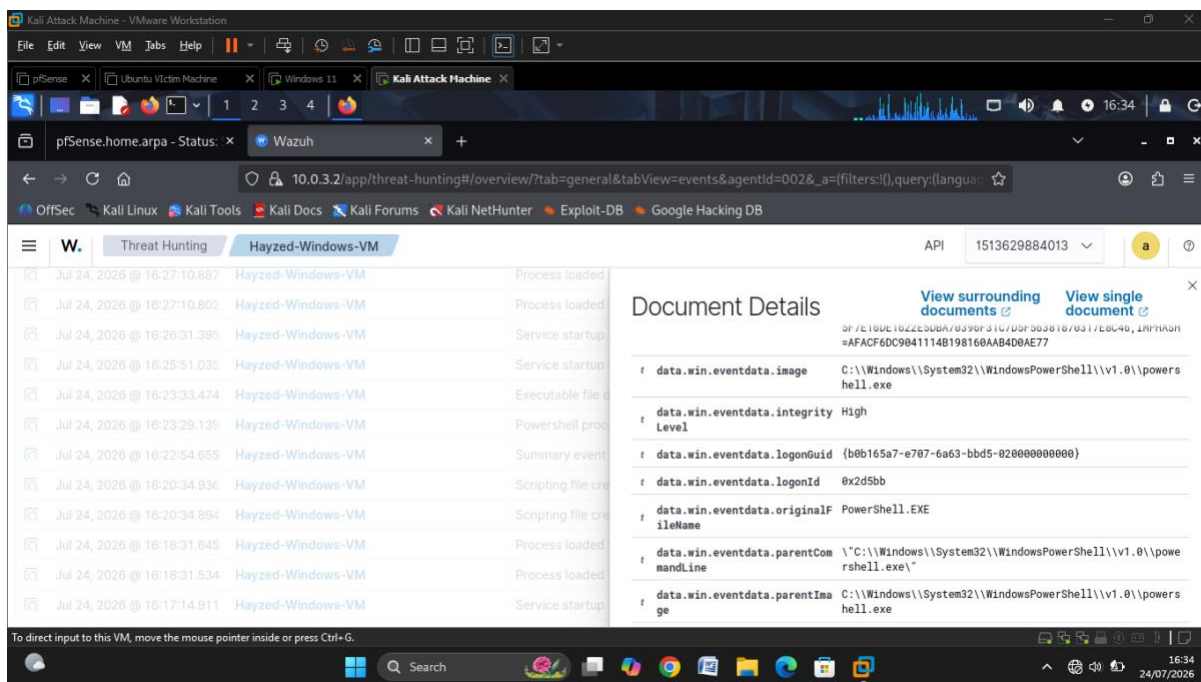


Figure 9: Sysmon events successfully collected in the Wazuh Dashboard.

pfSense Firewall Integration

pfSense Configuration

A pfSense firewall was deployed as the network log source for the SOC lab. Remote logging was enabled on the firewall, and syslog events were configured to be forwarded to the Wazuh server using the server's IP address and the standard syslog port. This allowed firewall events to be transmitted to the SIEM for centralized monitoring.

Wazuh Configuration

The Wazuh Manager was configured to receive and process the forwarded firewall logs. Additional configuration changes were made to ensure that archived firewall logs were collected and made available for analysis.

Log Parsing

To improve the readability of the firewall events, a custom decoder and detection rule were created within Wazuh. The decoder extracted key information from the pfSense syslog messages, while the custom rule identified blocked firewall traffic and generated a descriptive security alert whenever a connection was denied.

Validation

The integration was validated by generating network traffic through the pfSense firewall. An SSH connection attempt was made from a system outside the protected network to an IP address located behind the pfSense firewall. The firewall blocked the connection attempt and forwarded the corresponding syslog event to Wazuh. Wazuh successfully received, parsed, and generated an alert for the blocked traffic, confirming that firewall log forwarding and alerting were functioning correctly.

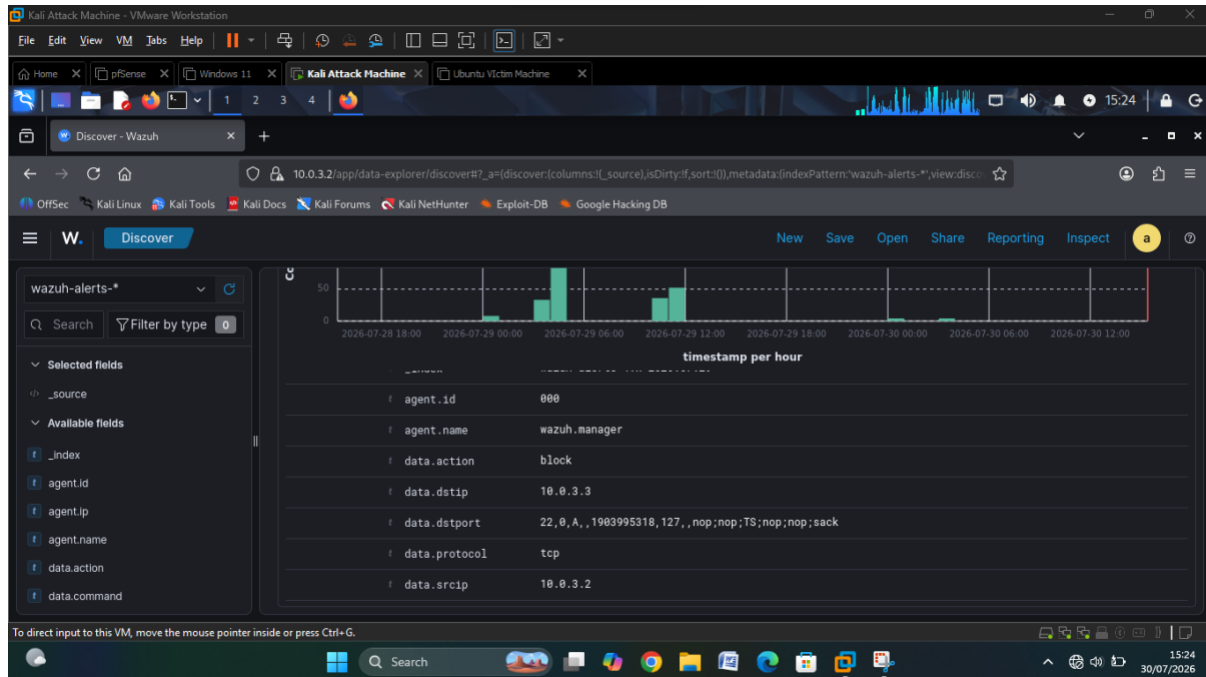


Figure 10: Blocked SSH connection attempt detected in the Wazuh Dashboard. Wazuh Dashboard.

Dashboard Development

To improve security monitoring and provide a centralized view of security events, three dashboards were created in the Wazuh Dashboard using the built-in visualization tools. Each dashboard was populated with live data generated within the SOC lab to support monitoring and incident analysis.

Authentication Activity Dashboard

The Authentication Activity Dashboard was developed to monitor user authentication events across the lab environment. It includes visualizations showing successful and failed login attempts over time, authentication activity grouped by monitored host, and failed login attempts grouped by user account. These visualizations make it easier to identify unusual authentication patterns and potential brute-force attacks.

Network Traffic Overview Dashboard

The Network Traffic Overview Dashboard was built using pfSense firewall logs collected by Wazuh. The dashboard displays network traffic trends, the most active source and destination IP addresses, and the distribution of allowed and blocked network connections. These visualizations provide an overview of network activity and assist in identifying abnormal traffic patterns.

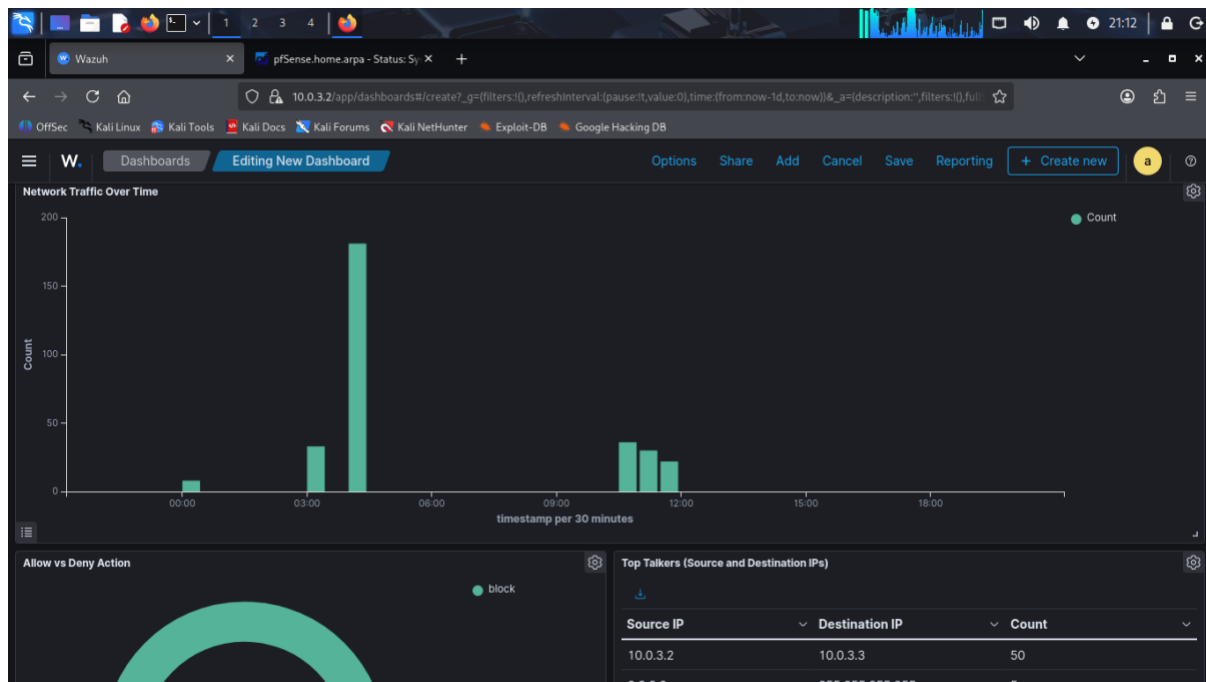


Figure 13: Network Traffic Overview Dashboard.

Custom Detection Rules

To improve Wazuh's detection capabilities, two custom correlation rules were developed. These rules were designed to identify attack patterns that were not fully addressed by the default detection rules and to generate alerts when suspicious behaviour was observed.

Brute-Force Login Detection Rule

A custom correlation rule was created to detect potential brute-force attacks against the Ubuntu SSH service. The rule was configured to monitor failed SSH authentication events and generate an alert when the same user account recorded **five failed login attempts within a five-minute period**. This approach helps distinguish repeated attack attempts from isolated login failures, reducing unnecessary alerts while improving the detection of password-guessing attacks.

Validation

The rule was validated by intentionally entering an incorrect password multiple times when attempting to log in to the Ubuntu virtual machine through SSH. After the configured threshold was reached, Wazuh generated a brute-force alert, confirming that the correlation rule was functioning as expected.

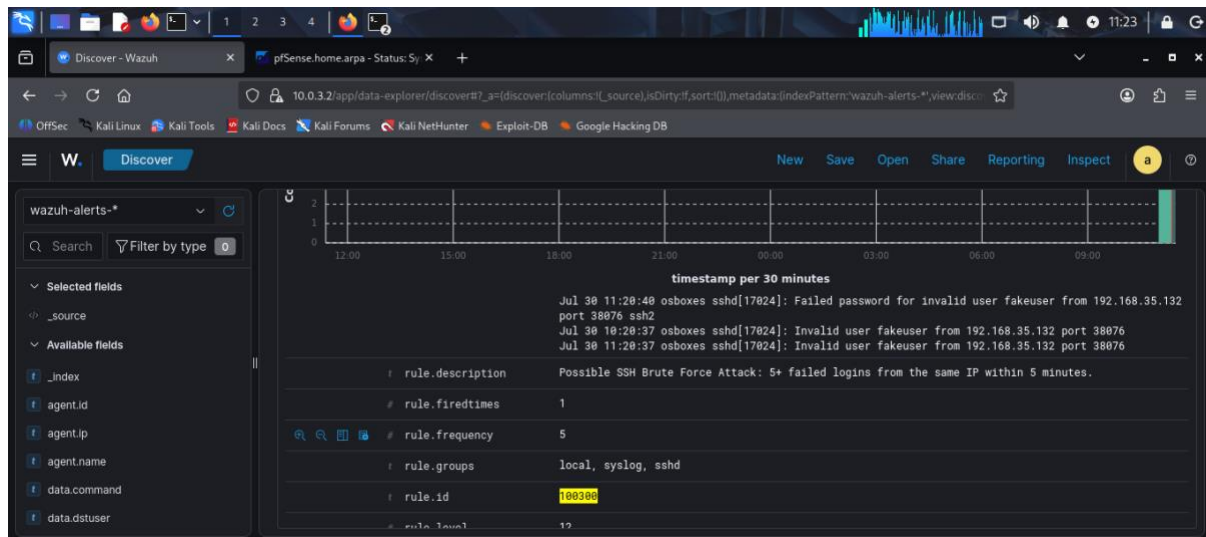


Figure 14: Custom brute-force detection alert generated by Wazuh.

Suspicious Process Execution Rule

A second custom rule was created to detect processes executed from directories that are commonly associated with malicious activity, such as temporary folders. The rule was configured to monitor process creation events collected from the Windows endpoint and generate an alert whenever an executable was launched from a suspicious location.

Validation

The rule was validated by executing a test application from a monitored temporary directory. Wazuh successfully detected the activity and generated the expected alert, confirming that the custom rule correctly identified suspicious process execution.

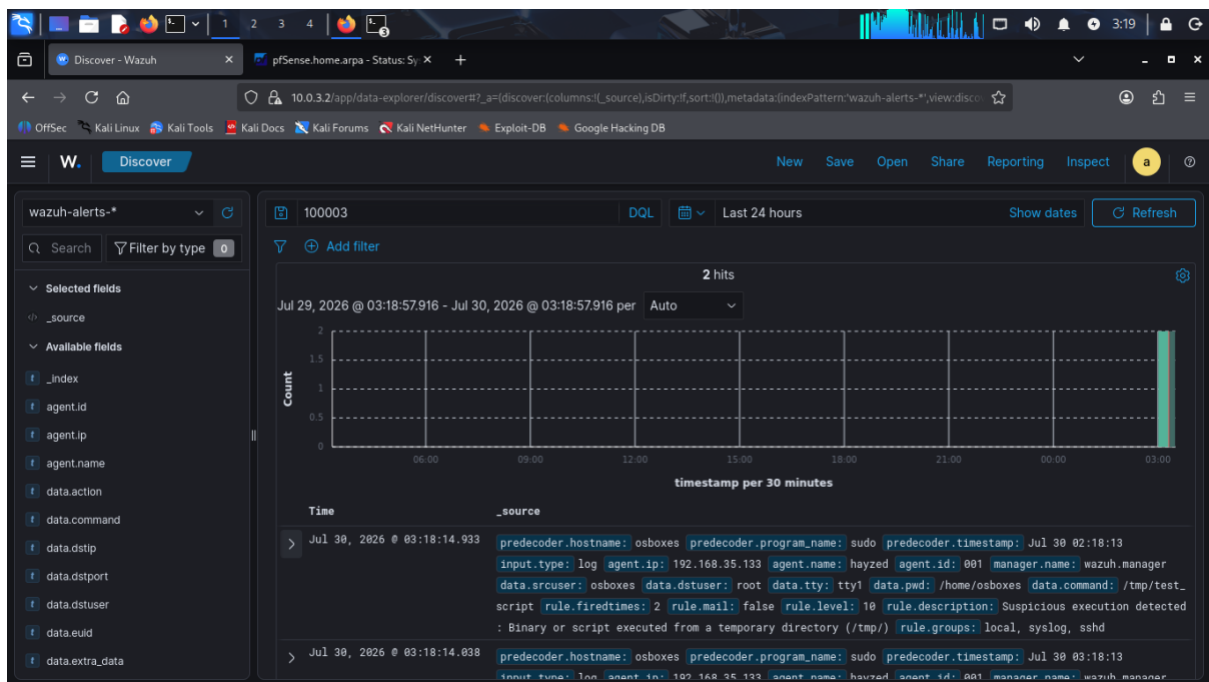


Figure 15: Suspicious process execution alert generated by Wazuh.

Findings

Authentication Activity Dashboard

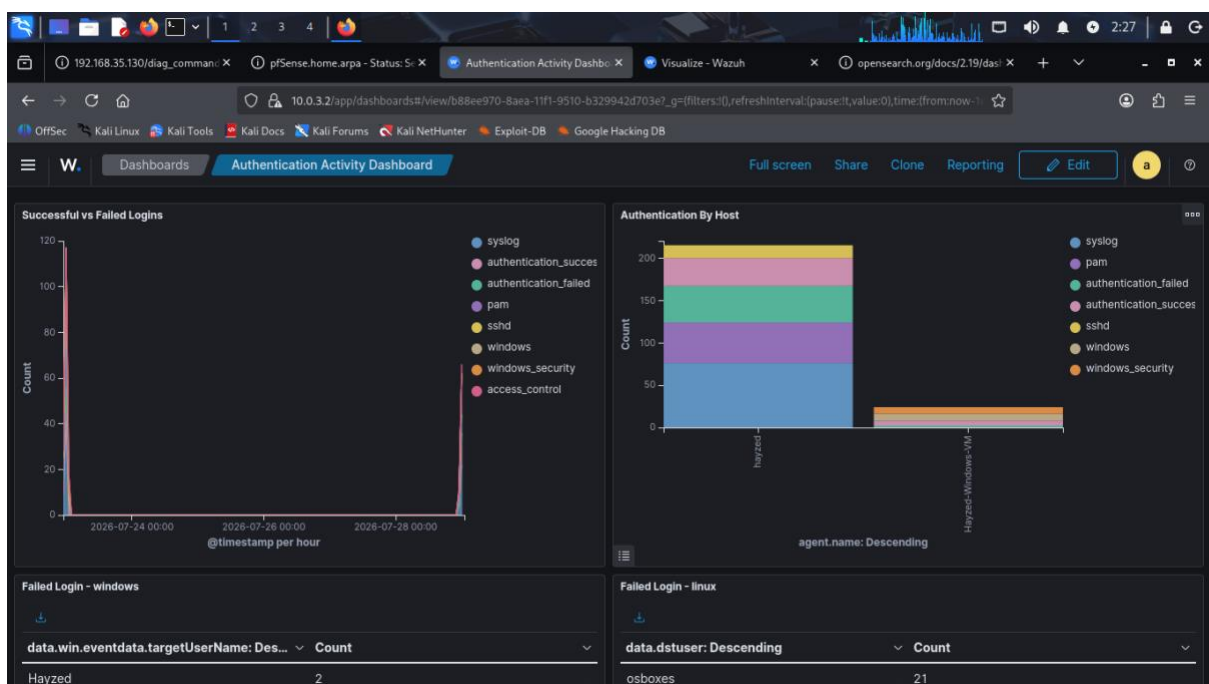


Figure 16: *Authentication Activity Dashboard*

The Authentication Activity Dashboard provides a centralized view of authentication events across the SOC lab. It displays successful and failed login attempts over time, authentication activity grouped by monitored host, and failed login attempts by user account. During testing, the dashboard successfully captured the failed login attempts generated on the Ubuntu endpoint, confirming that authentication events were being collected and visualized correctly.

Endpoint Process Activity Dashboard

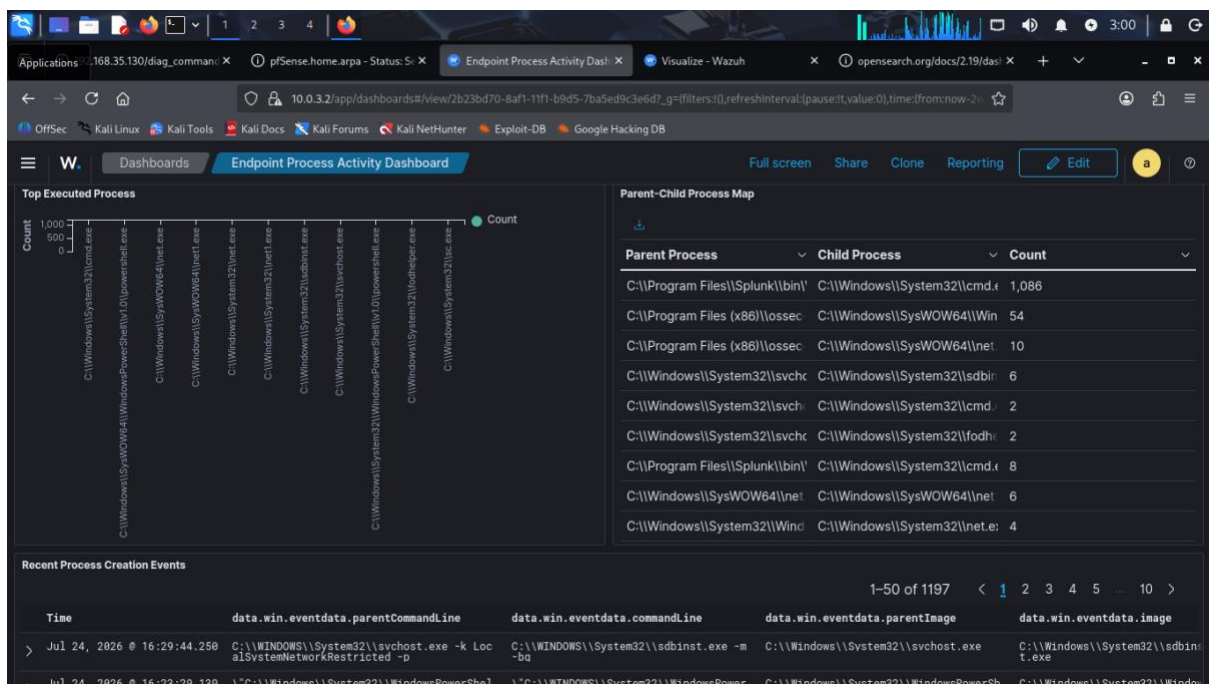


Figure 17: Endpoint Process Activity Dashboard

The Endpoint Process Activity Dashboard provides visibility into process execution on the Windows endpoint using Sysmon logs. It displays the most frequently executed processes, parent-child process relationships, and a live feed of recent process creation events. This dashboard confirmed that Sysmon events were successfully collected and provided detailed endpoint activity for monitoring and investigation.

Network Traffic Overview Dashboard

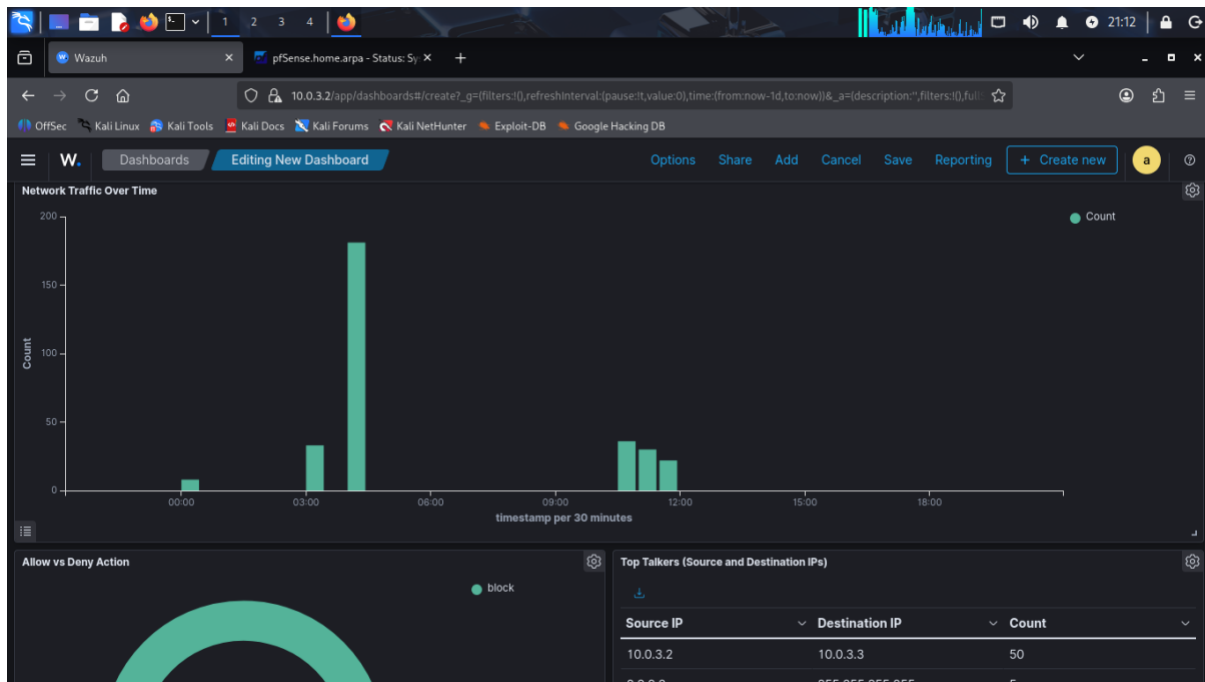


Figure 18: Network Traffic Overview Dashboard

The Network Traffic Overview Dashboard displays network activity collected from the pfSense firewall. It includes traffic volume over time, the most active source and destination IP addresses, and the distribution of allowed and blocked network connections. During testing, the dashboard successfully recorded the blocked SSH connection attempt, confirming that firewall logs were being received and visualized correctly.

Custom Correlation Rules

Brute-Force Login Detection Rule

A custom correlation rule was developed to detect repeated SSH authentication failures on the Ubuntu endpoint. The rule generates an alert when the same source IP address records **five failed login attempts within five minutes**, indicating a potential brute-force attack.

Validation

The rule was tested by intentionally attempting to log in to the Ubuntu virtual machine via SSH using invalid credentials multiple times. After the fifth failed login attempt, Wazuh generated the custom brute-force alert, confirming that the rule functioned as intended.

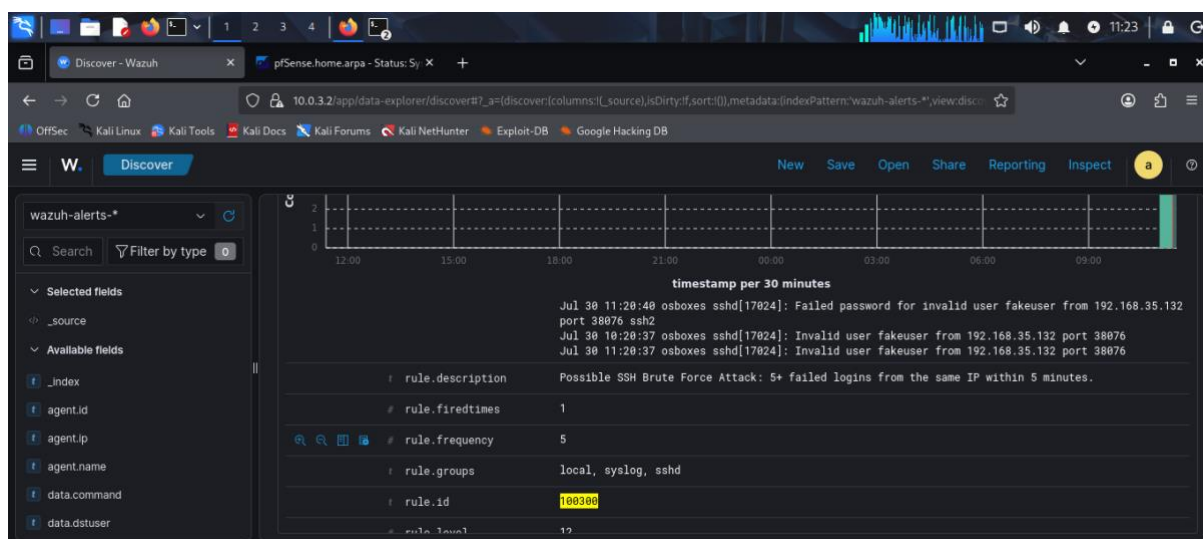


Figure 19: Custom brute-force detection alert.

Suspicious Process Execution Rule

A custom correlation rule was created to detect the execution of files or scripts from temporary directories, which are commonly abused by attackers. The rule monitors command execution and generates an alert whenever a binary or script is executed from the **/tmp/** directory.

Validation

The rule was validated by executing a test script from the **/tmp/** directory on the Ubuntu endpoint. Wazuh detected the activity and generated the custom alert, confirming that suspicious process execution was successfully identified.

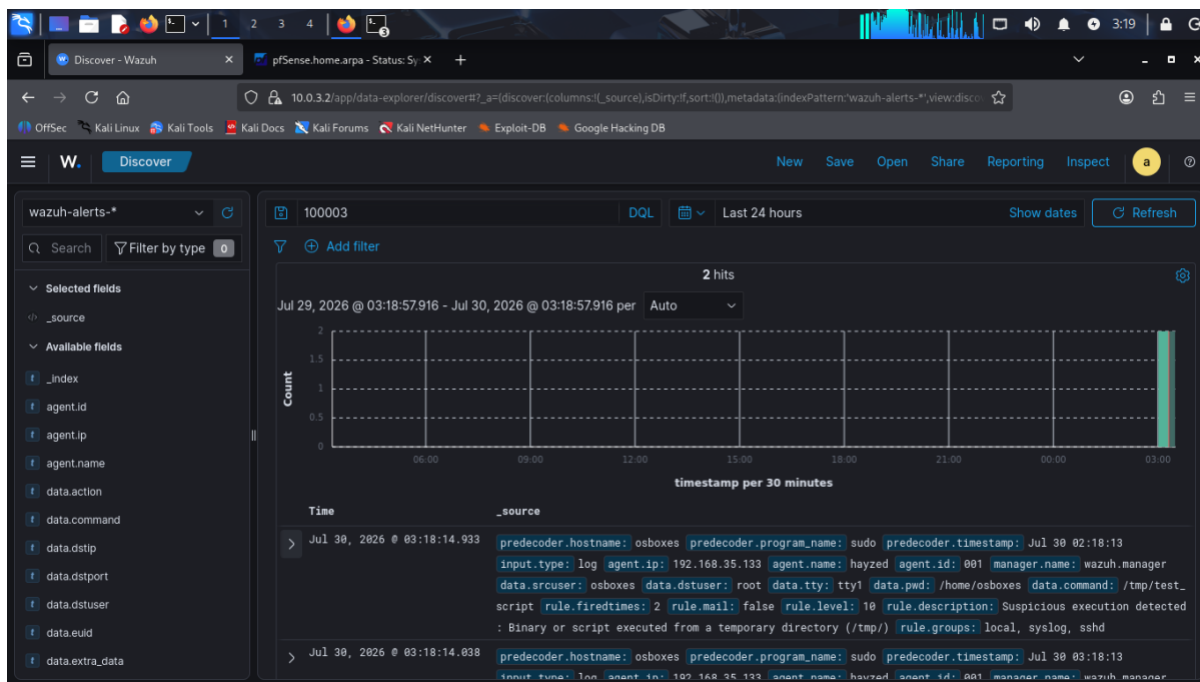


Figure 20: Suspicious process execution alert.

Gaps and Challenges

Wazuh Installation

The biggest challenge encountered during the project was deploying the Wazuh platform using the standard installation method. The installation repeatedly failed because the Wazuh Dashboard could not register with the Wazuh API, causing the installation to roll back automatically. After researching alternative deployment methods, the official Docker deployment was used instead. Although Docker requires configuration changes to be made from within the containers, it provided a more stable deployment and allowed memory resources to be allocated individually to the Wazuh Manager, Indexer, and Dashboard.

Limited System Resources

The project was completed on a system with **8 GB of RAM**, which made it difficult to run multiple virtual machines simultaneously. To overcome this limitation, the Ubuntu virtual machine was converted from a graphical desktop environment to a command-line server. This reduced its memory usage, allowing the endpoint to run the Wazuh agent and Suricata while freeing additional resources for the rest of the SOC lab.

pfSense Network Configuration

During the initial deployment of pfSense, only one network adapter was configured. As a result, the firewall could not establish connectivity to the internet or the required update services. The issue was resolved by adding a second network adapter configured for the WAN network while retaining the isolated LAN network for the SOC environment.

Agent Connectivity

Another challenge occurred when the Wazuh agents were unable to communicate with the Wazuh server whenever pfSense was not running. This was resolved by creating an additional NAT network that enabled reliable communication between the monitored endpoints and the SIEM server while maintaining the isolated lab environment.

Windows Authentication Logs

Initially, failed Windows login attempts were not appearing in Wazuh because Windows was not configured to audit failed logon events. This was resolved by enabling **Logon Failure Auditing** through the Local Security Policy, after which failed authentication events were successfully collected by the SIEM.

pfSense Log Integration

Configuring pfSense to forward firewall logs to Wazuh required additional troubleshooting before the logs were successfully parsed and displayed in the dashboard. After researching the issue and adjusting the required configurations, firewall events were successfully collected and visualized.

Brute-Force Detection Rule

The custom brute-force detection rule did not initially trigger as expected because it referenced an incorrect event ID. After reviewing the rule logic and modifying it to reference the correct matching rule ID, the correlation rule successfully detected five failed SSH login attempts within the configured time window.

Limitations

Although the SOC lab provides centralized monitoring of Windows, Ubuntu, and pfSense logs, it does not currently include cloud log sources, DNS monitoring, or email security telemetry. Integrating these additional log sources would improve visibility across the environment and provide broader detection coverage for future enhancements.

Recommendations and Next Steps

Given additional time, several improvements could be made to enhance the SOC lab and expand its monitoring capabilities.

Integrate DNS Monitoring

Adding DNS logs would improve visibility into domain name resolution activity and help detect suspicious or malicious domain requests that may indicate malware communication or command-and-control traffic.

Integrate Cloud Log Sources

The current SOC lab focuses on on-premises systems. Future work could include integrating cloud platforms such as Microsoft Azure or Amazon Web Services (AWS) to monitor cloud resources alongside traditional endpoints.

Expand Threat Intelligence Integration

While VirusTotal was successfully integrated for file reputation checks, additional threat intelligence feeds could be incorporated to improve the identification of malicious IP addresses, domains, and file hashes.

Develop Additional Detection Rules

The current implementation includes custom rules for brute-force login attempts and suspicious process execution. Future enhancements could include detection rules for privilege escalation, persistence techniques, PowerShell abuse, and lateral movement to improve overall threat detection coverage.

Automate Additional Response Actions

The current active response automatically removes files confirmed as malicious by VirusTotal. Future work could extend automation to include actions such as blocking malicious IP addresses at the firewall, isolating compromised endpoints, or disabling compromised user accounts.

Improve Dashboard Coverage

Additional dashboards could be developed to monitor Suricata intrusion detection events, VirusTotal detections, and system integrity monitoring. This would provide analysts with greater visibility into different areas of the environment from a single interface.

Appendix

A. Custom Wazuh Correlation Rules (`local_rules.xml`)

Rule 1 – SSH Brute Force Detection

```
<rule id="100300" level="12" frequency="5"
timeframe="300">
```

```
  <if_matched_sid>5710</if_matched_sid>
```

```
  <same_source_ip />
```

```
  <description>Possible SSH Brute Force Attack: 5+ failed
logins from the same IP within 5 minutes.</description>
```

```
  <mitre>
```

```
    <id>T1110</id>
```

```
  </mitre>
```

```
</rule>
```

Purpose: Detects five or more failed SSH login attempts from the same source IP within five minutes.

Rule 2 – Suspicious Process Execution

```
<rule id="100003" level="10">
```

```
  <if_sid>5402,5406,5501,1002,2901,2902</if_sid>
```

```
  <regex>/tmp|var/tmp</regex>
```

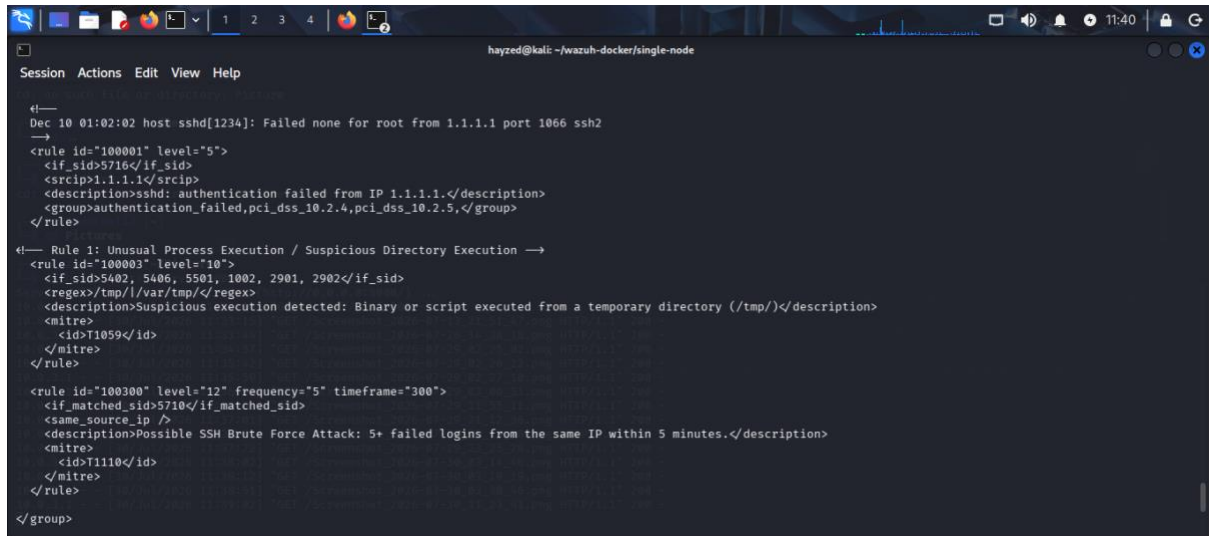
```
  <description>Suspicious execution detected: Binary or script executed from a temporary
directory (/tmp).</description>
```

```
  <mitre>
```

```
    <id>T1059</id>
```

```
  </mitre>
```

</rule>



```
Session Actions Edit View Help
hayzed@kali: ~/wazuh-docker/single-node

Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
<rule id="100001" level="5">
  <if_sid>5716</if_sid>
  <srcip>1.1.1.1</srcip>
  <description>sshd: authentication failed from IP 1.1.1.1.</description>
  <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>

Rule 1: Unusual Process Execution / Suspicious Directory Execution
<rule id="100003" level="10">
  <if_sid>5402, 5406, 5501, 1002, 2901, 2902</if_sid>
  <regex>/tmp/|/var/tmp/</regex>
  <description>Suspicious execution detected: Binary or script executed from a temporary directory (/tmp/)</description>
  <mitre>
    <id>T1059</id>
  </mitre>
</rule>

<rule id="100300" level="12" frequency="5" timeframe="300">
  <if_matched_sid>5710</if_matched_sid>
  <same_source_ip />
  <description>Possible SSH Brute Force Attack: 5+ failed logins from the same IP within 5 minutes.</description>
  <mitre>
    <id>T1110</id>
  </mitre>
</rule>
</group>
```

Purpose: Detects binaries or scripts executed from temporary directories, which are commonly abused by attackers.

B. Custom pfSense Decoder (local_decoder.xml)

<decoder name="pfsense">

<program_name>filterlog</program_name>

</decoder>

<decoder name="pfsense-details">

<parent>pfsense</parent>

<regex>\d+,\d+,(.*),(.*),(.*),(.*),(.*),(.*),(.*),(.*)</regex>

<order> Action, interface, reason, action_type, ip_version, src_ip, dst_ip, proto, length, src_port, dst_port </order>

</decoder>

Purpose: Parses pfSense firewall logs into structured fields for analysis and dashboard visualization.


```
hayzed@kali: ~/wazuh-docker/single-node
Session Actions Edit View Help
<description>Possible SSH Brute Force Attack: 5+ failed logins from the same IP within 5 minutes.</description>
<mitre>
  <id>T1110</id>
</mitre>
</rule>
</group>
<group name="syscheck,pci_dss_11.5,nist_800_53_SI.7,">
  <rule id="100200" level="7">
    <if_sid>554</if_sid>
    <match>^/root!~/home!~/tmp!~/var/tmp</match>
    <description>File added to suspicious directory</description>
  </rule>
  <rule id="100201" level="7">
    <if_sid>550</if_sid>
    <match>^/root!~/home!~/tmp!~/var/tmp</match>
    <description>File modified in suspicious directory</description>
  </rule>
</group>
<group name="pfsense,firewall,">
  <rule id="100099" level="7">
    <match>blocks</match>
    <description>pfsense: Firewall Blocked Traffic</description>
  </rule>
</group>
(hayzed@kali) - [~/wazuh-docker/single-node]
$
```

D. Windows Agent Configuration

The Windows Wazuh agent was configured to collect:

- Windows Security Event Logs
- Microsoft Sysmon Operational Logs
- Active Response Logs

```
<localfile>
```

```
<location>Microsoft-Windows-Sysmon/Operational</location>
```

```
<log_format>eventchannel</log_format>
```

```
</localfile>
```

E. Sysmon Installation (after downloading on file manager)

Sysmon was installed on the Windows endpoint using:

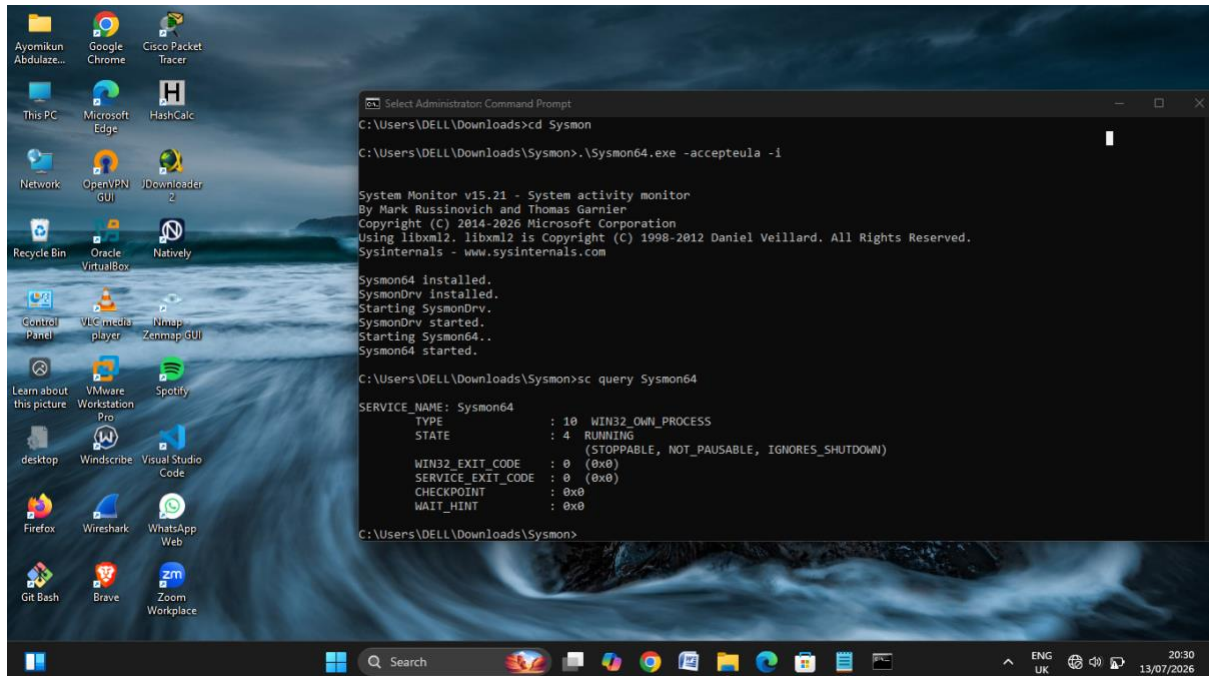
```
.\Sysmon64.exe -accepteula -i
```

Verification:

```
sc query Sysmon64
```

Status:

STATE : RUNNING

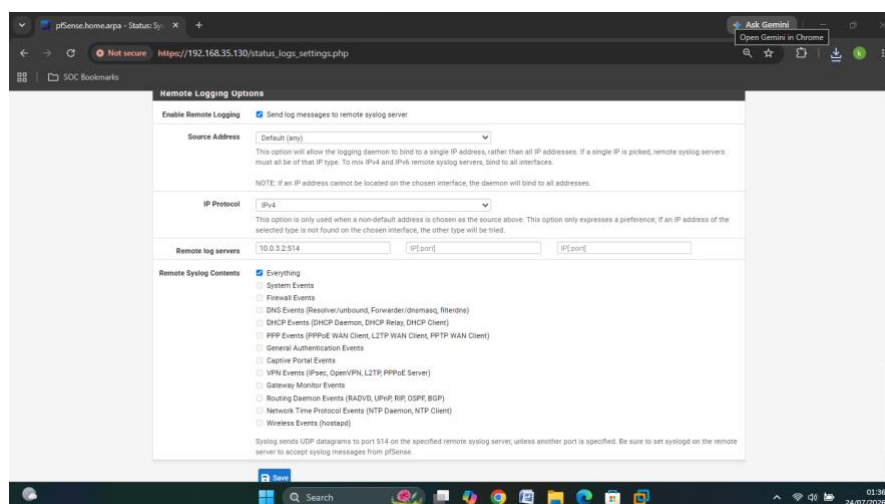


F. pfSense Remote Syslog Configuration

Remote logging configuration:

- Remote Syslog Enabled
- Destination: 10.0.3.2:514
- Protocol: UDP
- Log Content: Everything

This allowed pfSense firewall logs to be forwarded to the Wazuh manager for analysis.



G. Wazuh Docker Remote Syslog Configuration

<remote>

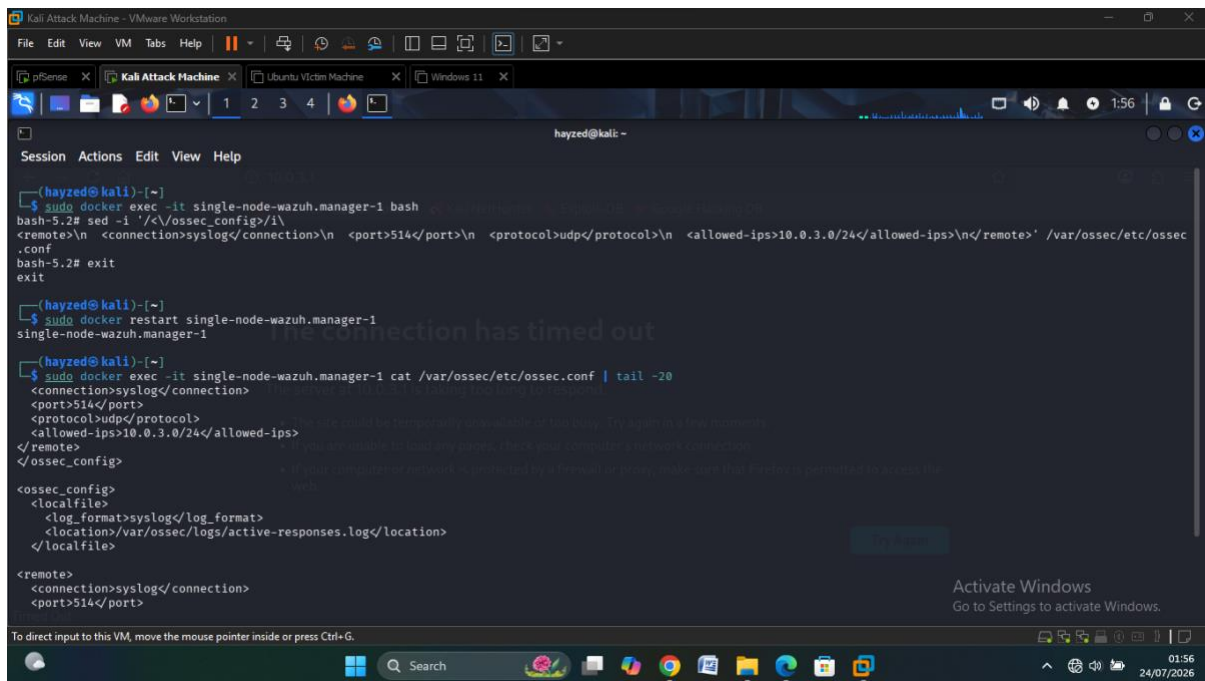
<connection>syslog</connection>

<port>514</port>

<protocol>udp</protocol>

<allowed-ips>10.0.3.0/24</allowed-ips>

</remote>



```
(hayzed@kali)-[~]
$ sudo docker exec -it single-node-wazuh.manager-1 bash
bash-5.2# sed -i '/</ossec_config>/I'
<remote>\n  <connection>syslog</connection>\n  <port>514</port>\n  <protocol>udp</protocol>\n  <allowed-ips>10.0.3.0/24</allowed-ips>\n</remote>' /var/ossec/etc/ossec
.conf
bash-5.2# exit
exit

(hayzed@kali)-[~]
$ sudo docker restart single-node-wazuh.manager-1
single-node-wazuh.manager-1

(hayzed@kali)-[~]
$ sudo docker exec -it single-node-wazuh.manager-1 cat /var/ossec/etc/ossec.conf | tail -20
<connection>syslog</connection>
<port>514</port>
<protocol>udp</protocol>
<allowed-ips>10.0.3.0/24</allowed-ips>
</remote>
</ossec_config>
<ossec_config>
  <localfile>
    <log_format>syslog</log_format>
    <location>/var/ossec/logs/active-responses.log</location>
  </localfile>
</remote>
<connection>syslog</connection>
<port>514</port>
```